

Dockerisation de l'architecture microservices



Formation : Applications Web Distribuées



Ons Ben Amor



Année universitaire 2025-2026



Plan de formation

Dockerisation de l'architecture microservices

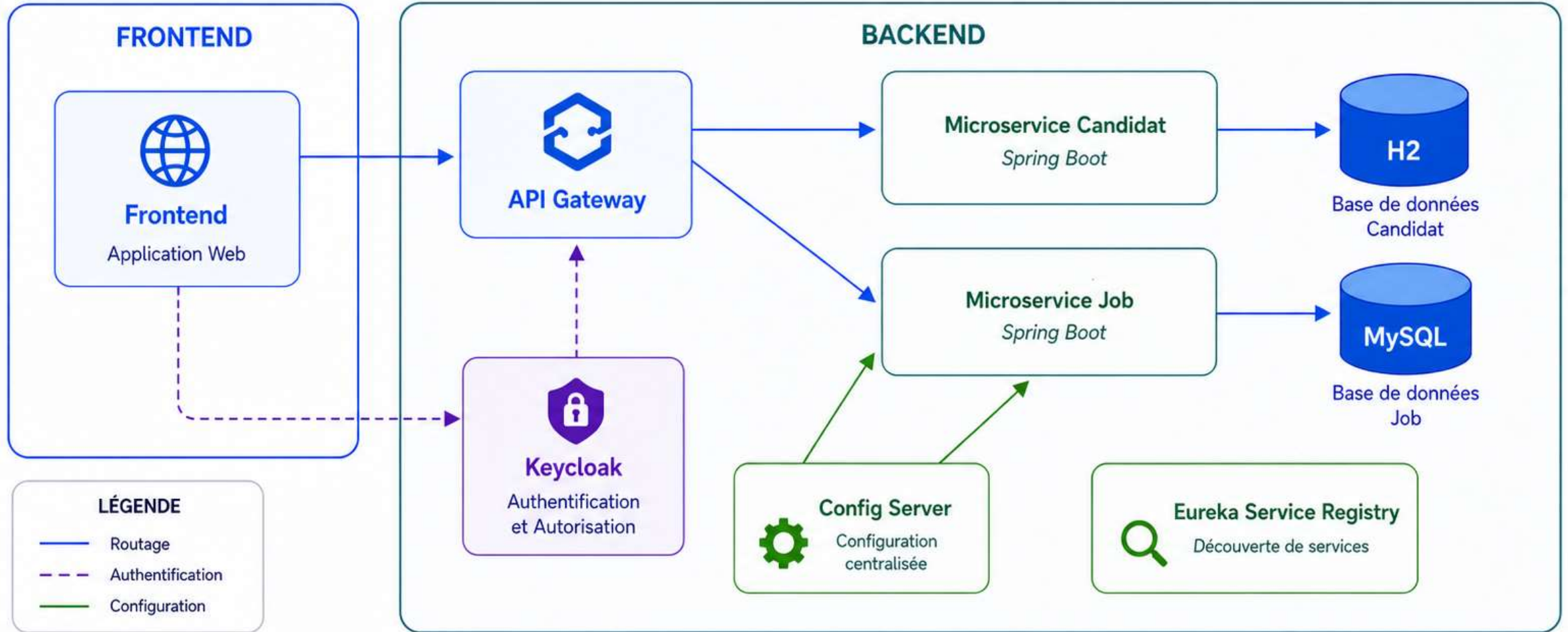
- 01 Architecture dockerisée
- 02 Introduction à Docker
- 03 Atelier Dockerfile / Build / Run
- 04 Atelier Docker Compose
- 05 Docker Hub & KillerCoda





Environnement Docker

docker-compose.yml

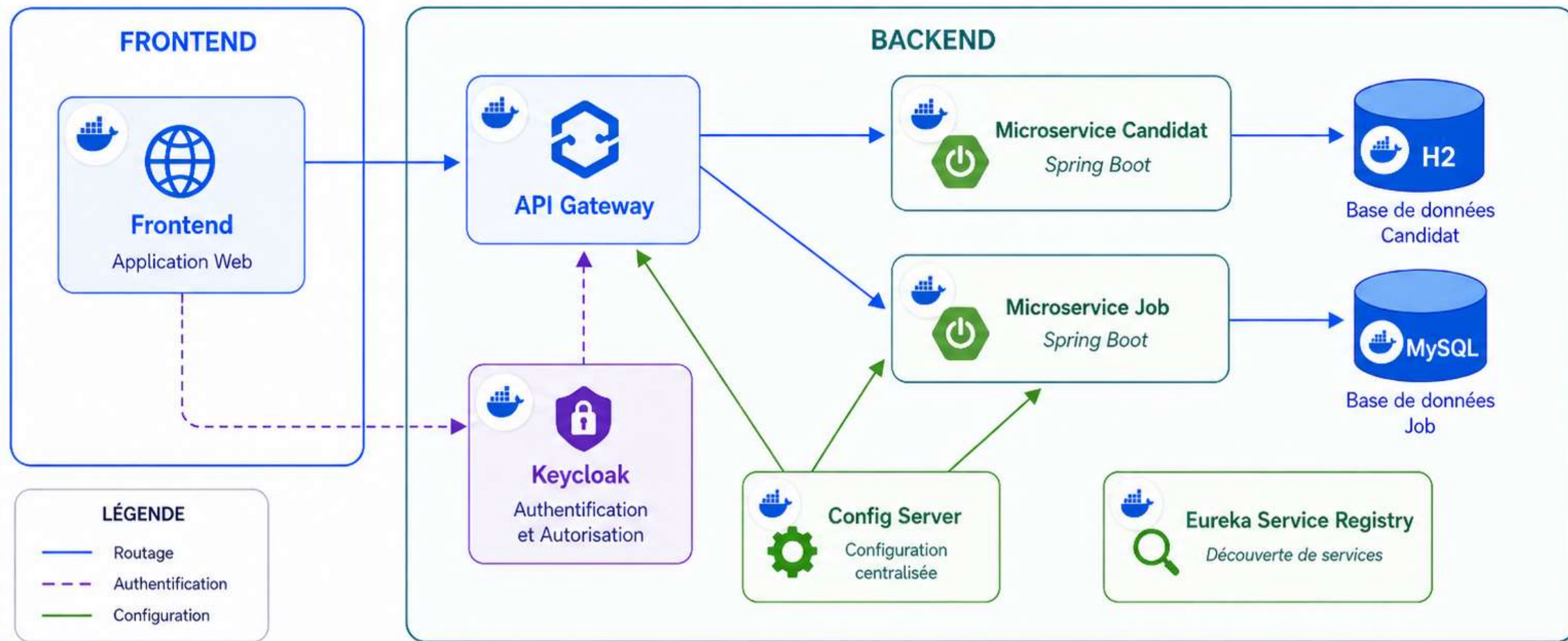


Tous les services sont lancés par Docker Compose



Environnement Docker

docker-compose.yml



Tous les services sont lancés par Docker Compose

Qu'est-ce que Docker ?



Docker est un outil qui permet de regrouper une application et tout ce dont elle a besoin dans un conteneur, afin de l'exécuter facilement et de manière fiable dans différents environnements.

Microservices + Docker

Une application devient un ensemble de services conteneurisés : plus faciles à lancer, déplacer, tester et maintenir.

Pourquoi Docker avec les microservices ?

1

Environnement normalisé

Chaque service embarque ses dépendances : même comportement en dev, test et production.

2

Développement accéléré

Chaque équipe lance un service isolé sans installer tout l'écosystème.

3

Portabilité améliorée

Une image peut être déplacée vers une autre machine ou vers le cloud.

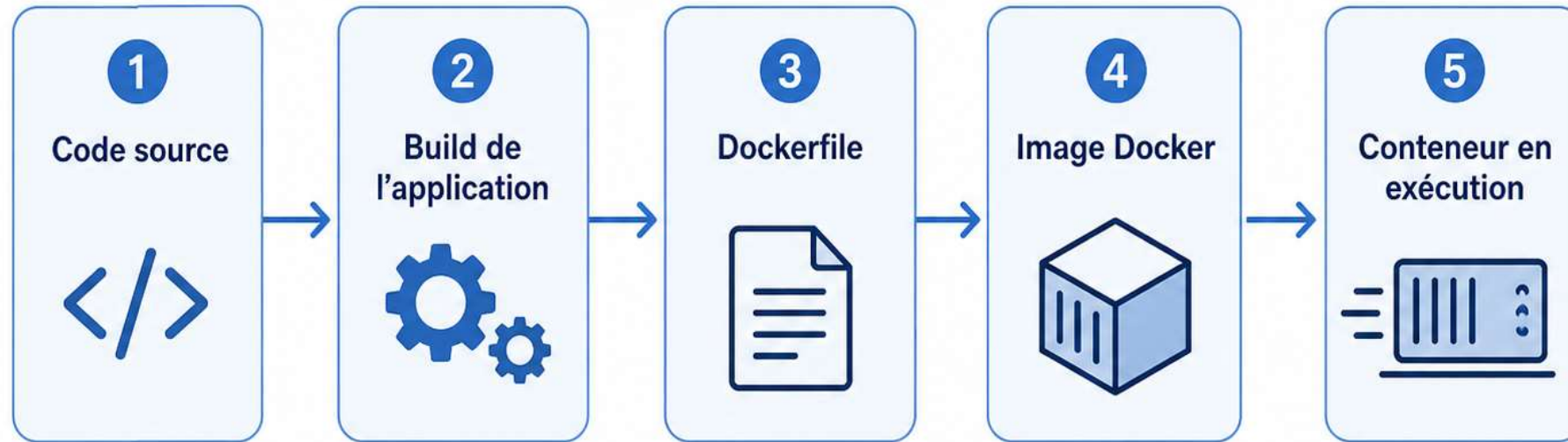
4

Réduction des ressources

Les containers consomment moins qu'une machine virtuelle complète.

Conteneurisation d'un microservice

Schéma général sans technologie spécifique



Idée clé

Même logique pour tous les projets : code → build → image → conteneur.

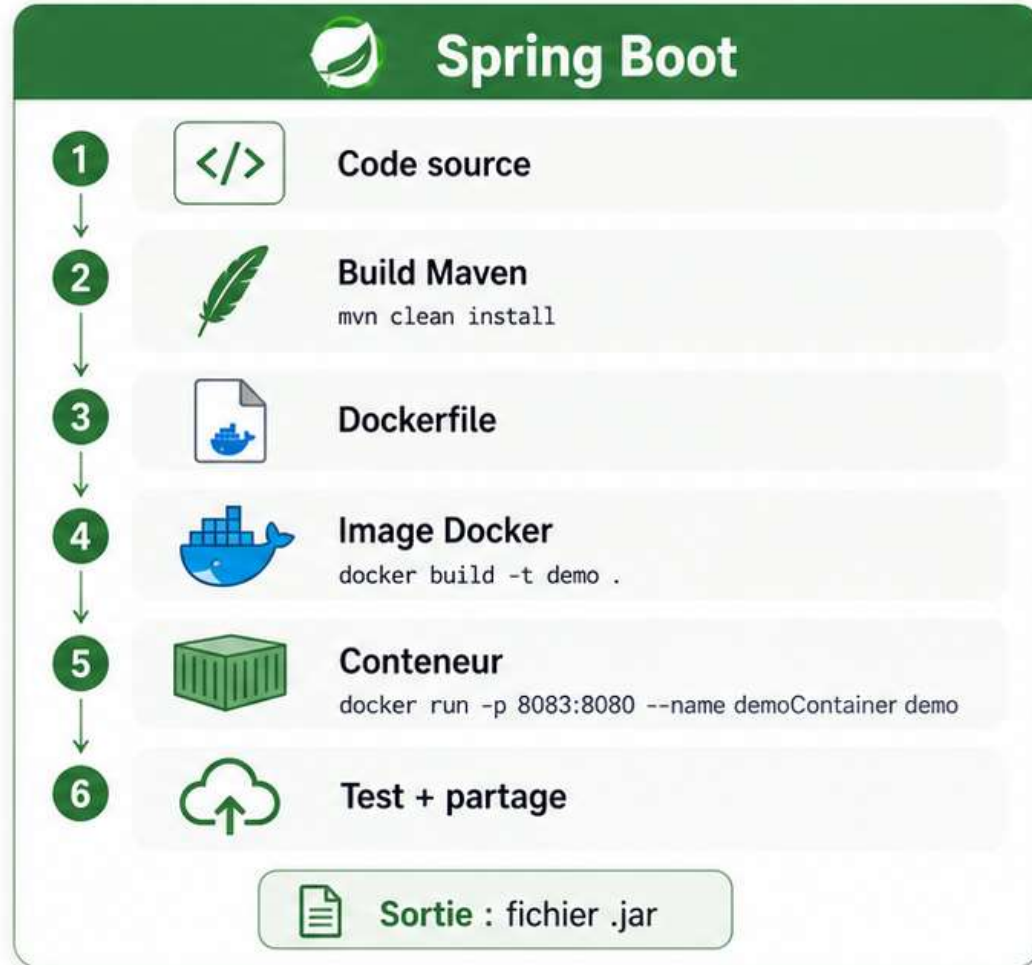


Résultat

L'application est emballée avec ses dépendances et peut être exécutée de manière cohérente dans différents environnements.

Conteneurisation d'un microservice : Spring Boot vs NestJS

Même logique Docker, technologies différentes



Transition vers les ateliers Docker

Ce que les participants doivent savoir avant la pratique

Atelier 1 — Docker Build

- Créer un Dockerfile à la racine du microservice.
- Générer le .jar avec Maven.
- Construire une image avec docker build.
- Démarrer un container avec docker run.
- Tester l'API puis pousser l'image sur Docker Hub.

Atelier 2 — Docker Compose

- Décrire toute la stack dans docker-compose.yml.
- Automatiser build, run, ports, volumes et dépendances.
- Lancer plusieurs containers avec une seule commande.
- Intégrer Gateway, Eureka, Config Server, Keycloak, MySQL et microservices.
- Tester localement puis sur KillerCoda avec les images Docker Hub.

Atelier 1

Dockerfile : transformer un microservice Spring Boot en image

Le Dockerfile est le plan utilisé par docker build.



```
FROM eclipse-temurin:17
EXPOSE 8080
ADD target/Candidat4SAE2-0.0.1-SNAPSHOT.jar candidat.jar
ENTRYPOINT ["java", "-jar", "candidat.jar"]
```

Dockerfile de l'atelier



Lecture du Dockerfile

- FROM eclipse-temurin:17 : base Java 17.
- EXPOSE 8080 : port interne du MS.
- ADD target/*.jar : copie le jar dans l'image.
- ENTRYPOINT : commande lancée au démarrage.

Exemple minimal

```
FROM eclipse-temurin:17
EXPOSE 8080
ADD target/Candidat-0.0.1-SNAPSHOT.jar candidat.jar
ENTRYPOINT ["java", "-jar", "candidat.jar"]
```



Point clé

- Avant docker build, Maven doit générer le fichier .jar dans le dossier target/.

Atelier 1 — Construire l'image Docker

Docker lit le Dockerfile et produit une image locale.

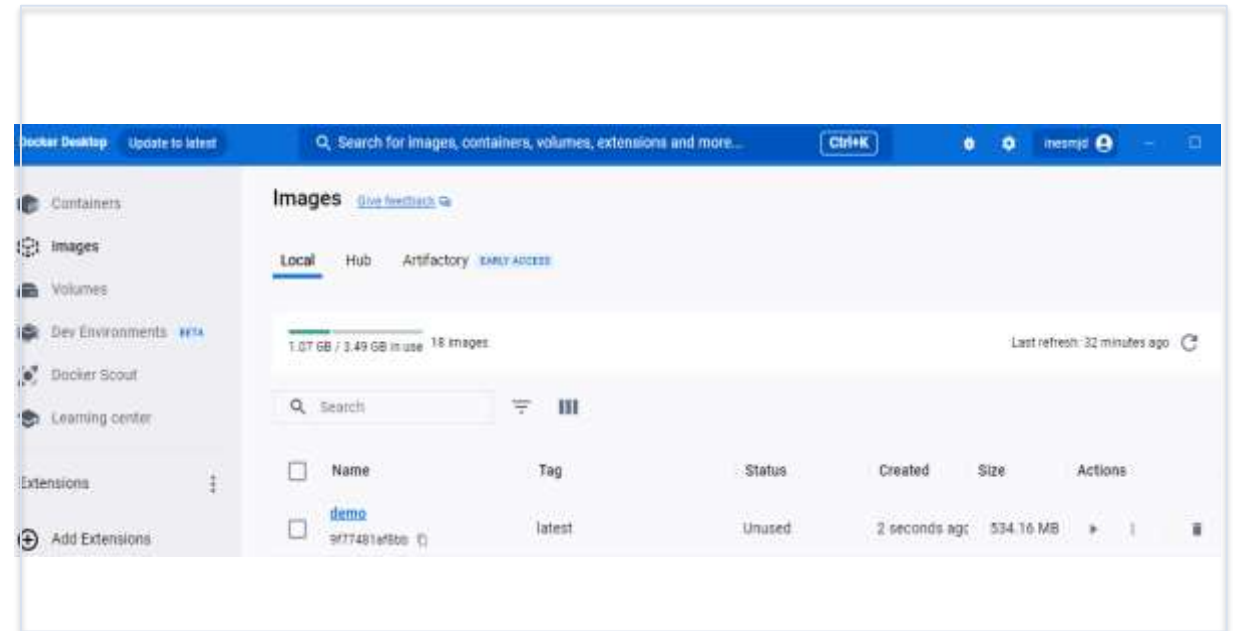
Commande terminal

```
docker build -t demo .
```

```
Build Log Dashboard
Step 1/4 : FROM openjdk:17
----> 5e28ba2b4cdb
Step 2/4 : EXPOSE 8080
----> Running in 545cca4409d1
Removing intermediate container 545cca4409d1
----> d88ebd184a89
Step 3/4 : ADD target/Candidat-0.0.1-SNAPSHOT.jar Candidat.jar
----> a8bca84a7d2e
Step 4/4 : ENTRYPOINT ["java", "-jar", "Candidat.jar"]
----> Running in 6670015fa272
Removing intermediate container 6670015fa272
----> 9f77481af8bb

Successfully built 9f77481af8bb
Successfully tagged demo:latest
'demo Dockerfile: Dockerfile' has been deployed successfully.
```

Build réussi



Docker Desktop : image locale

Interprétation

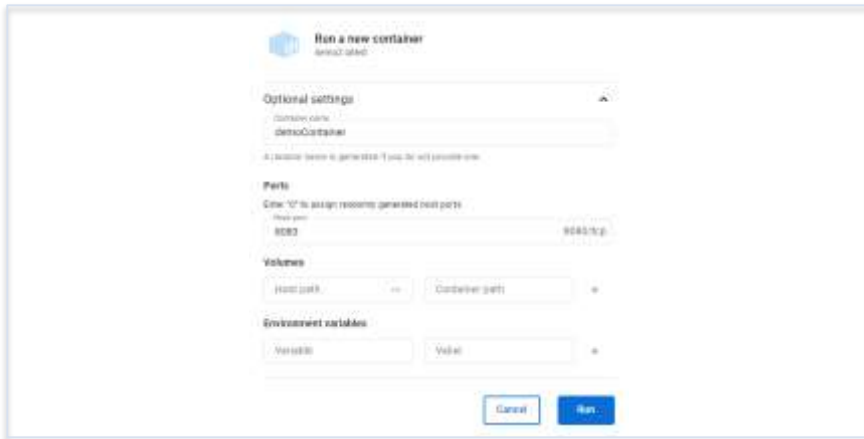
- -t demo : nom de l'image.
- . : Dockerfile du dossier courant.
- Image locale : pas encore publiée sur Docker Hub.

Atelier 1 — Démarrer et tester le container

Le container est l'instance exécutable de l'image.

Commande docker run

```
docker run -p 8083:8080 --name demoContainer demo
```



Run depuis Docker Desktop



Test navigateur : /candidates

□ Ce qu'il faut expliquer

- 8083 : port hôte.
- 8080 : port interne du container.
- demoContainer : nom du container.
- demo : image utilisée.

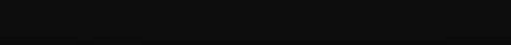


Résultat attendu

- L'URL `http://localhost:8083/candidates` doit retourner la liste JSON des candidats.

Atelier 1 — Docker Hub : partager l'image

Taguer puis pousser l'image pour la rendre accessible depuis une autre machine.



```

C:\Users\Muhammad Anwar\Downloads\CandidateProject (2)\CandidateProject>Bucker Login
Authenticating with existing credentials...
Login Successful

C:\Users\Muhammad Anwar\Downloads\CandidateProject (2)\CandidateProject>

```

docker login

```
C:\Users\Wahmed Akrin\Downloads\CandidateProject (2)\CandidateProject>docker login
Authenticating with existing credentials...
Login Succeeded

C:\Users\Wahmed Akrin\Downloads\CandidateProject (2)\CandidateProject>docker push dmw
Using default tag: latest
The push refers to repository [docker.io/library/dmw]
00aaf9400000: Pushed
00aaf9402000: Pushed
00aaf9402050: Pushing =====> 133MB/280.2MB
11a0b1c3a000: Pushed
00aaf9402070: Pushed
1944b0300000: Pushing =====> 2.1MB/56.183MB
1944b0300040: Pushed
b77420c0e700: Pushed
1.9.0: digest: sha256:ba72778cc890c96cc3a219c7b0abdd0f9a6a24f9561c8a71e95cffe9fbc size: 2807
```

tag + push



Images taguées

Commandes Docker Hub

```
docker login
docker tag demo <username>/<repository>:v1.0
docker push <username>/<repository>:v1.0
```

Pourquoi taguer ?

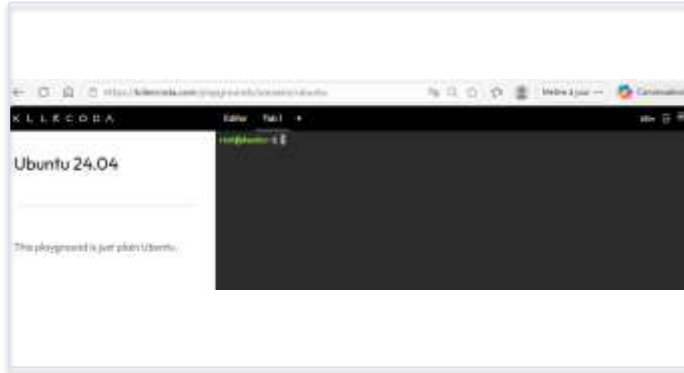
- Docker Hub attend username/repository:tag.
- Le tag permet de versionner : v1.0, latest.
- Sans tag, docker push peut échouer.

Bonne pratique

- Utiliser un tag clair par version d'atelier : v1.0, v1.1, etc.

Atelier 1 — Tester l'image dans KillerCoda

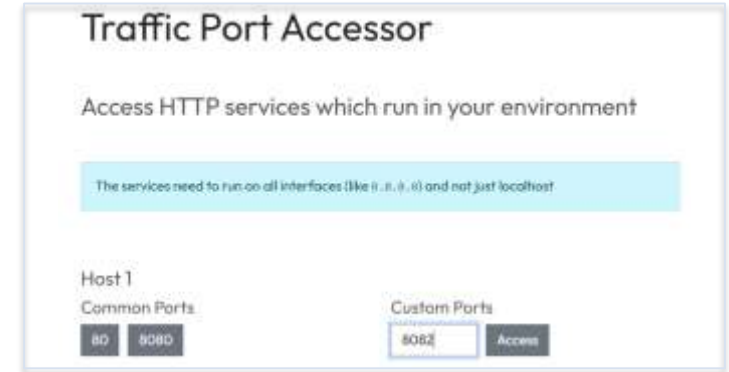
La machine cloud ne possède pas vos images locales : elle doit les télécharger depuis Docker Hub.



Playground Ubuntu KillerCoda



pull / run / ps



Traffic / Ports

Commandes cloud

```
docker pull <username>/<repository>:v1.0
docker run -d -p 8082:8080 --name candidat1 <username>/<repository>:v1.0
docker ps
curl http://localhost:8082/candidats
```

À expliquer

- Dans KillerCoda, localhost = machine cloud.
- Traffic / Ports permet d'ouvrir le service dans un navigateur.
- Le port 8082 redirige vers 8080 dans le container.

PARTIE 3 — ATELIER 2

Docker Compose : application multi-conteneurs

- Démarrer plusieurs services avec un seul fichier YAML
- Gérer les ports, dépendances et variables
- Inclure Keycloak, Eureka, Config Server, Gateway, Candidat, Job et MySQL

Pourquoi Docker Compose ?

Quand plusieurs microservices doivent fonctionner ensemble, docker run devient insuffisant.



Sans Compose

- Trop de commandes docker run.
- Ordre de démarrage difficile.
- Variables répétées.
- Réseau à configurer.



Avec Compose

- Un fichier docker-compose.yml.
- Une commande : docker compose up.
- Réseau commun entre services.
- Dépendances déclarées.

Résultat attendu

- Tous les containers démarrent ensemble.
- Les services s'enregistrent dans Eureka.
- Gateway route vers Candidat/Job.

Commandes Compose essentielles

```
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f
```



Positionnement

- Compose est idéal pour formation, local et test. Pour le cloud KillerCoda, on supprime build: et on utilise image:.

docker-compose.yml : structure du fichier

Le YAML décrit la stack complète : services, images, ports, dépendances et variables.

(Voir atelier!)

```

[api-gateway internal] boot_dockerize
-- >> transferring context: 38
[api-gateway internal] boot_build_context
-- >> transferring context: 604
CACHED [api-gateway 2/2] ADD target/takase-0.0.1-SNAPSHOT.jar gateway.jar
[api-gateway] exporting to image
-- >> exporting layers
-- >> exporting manifest sha256:111310c0f5b3a950a09c4971b89570190a7982358e-actff44290c870
-- >> exporting config sha256:daee6d3201c229e684c590c34e61f62090a4e572a811e0536041c1c5
-- >> exporting attestation manifest sha256:1a1633b6c8d0c8a6c793e6779033313f449030e3a44d6cc4c71ffac4
-- >> exporting manifest list sha256:7d71759a4e7708449021002c2e3094c3c645a51f5d5d080836d7707096
-- >> naming to docker.io/library/nginxgateway:1.0
-- >> pushing to docker.io/library/nginxgateway:1.0
[api-gateway] pushing procedure for attestation file
[+] Running 1/13
  ✓ api-gateway      0s/12t
  ✓ api-gat         0s/12t
  ✓ config-server    0s/12t
  ✓ eureka           0s/12t
  ✓ feign            0s/12t
  ✓ feign-microservices-interfaces_defaults Created
  ✓ container-eureka Created
  ✓ container-mysql Created
  ✓ container-keycloak Created
  ✓ container-configserver Created
  ✓ container-job Created
  ✓ container-candidat Created
  ✓ container-api-gateway Created

```

Extrait du docker-compose.yml

services

- containers à démarrer

image

- nom de l'image

depends_on

- dépendances

volumes

- données persistantes

build

- dossier du Dockerfile

ports

- hôte:container

environment

- variables

Règle critique

- Dans un container, localhost désigne le container lui-même.
- Pour appeler un autre service Compose, utiliser son nom : eureka-server, config-server, mysqldb, keycloak.

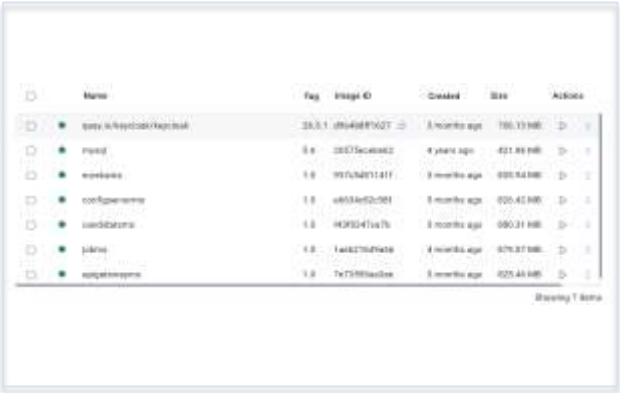
Atelier 2 — Lancer la stack multi-containers en local

Compose construit les images puis démarre les containers.

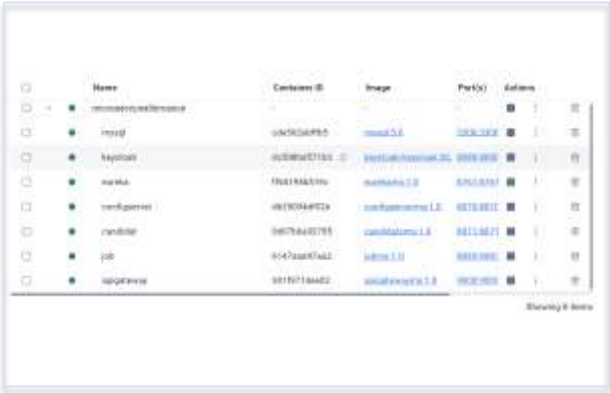
Lancement local

```
docker compose up --build
# ou en mode détaché
docker compose up -d --build

docker compose ps
docker ps
```



Images créées



Containers démarrés



Eureka : services enregistrés

Tests locaux

- Eureka : <http://localhost:8761/>
- Gateway : routes /candidats et /jobs
- Keycloak : <http://localhost:8099/>
- MySQL : port 3306 si besoin



Ordre de vérification

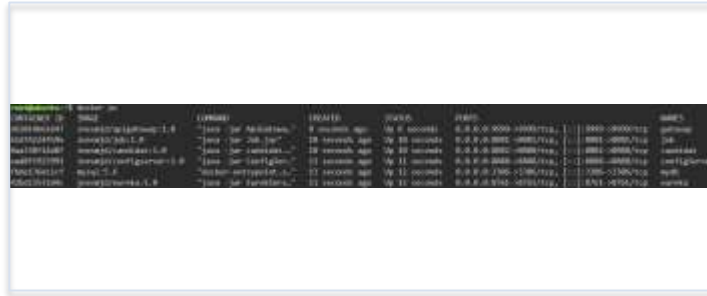
- 1) Tous les containers UP. 2) Eureka affiche les services. 3) Gateway route les requêtes. 4) Keycloak valide les tokens.

Atelier 2 — Exécuter la stack Compose dans KillerCoda

En cloud, Compose télécharge les images depuis Docker Hub.



docker compose build / push



docker compose up -d + docker ps



Traffic / Ports

Workflow cloud

```
# local
docker compose build
docker compose push

# cloud KillerCoda
vi docker-compose.yml
# supprimer les lignes build:
docker compose up -d
docker ps
```



Modification obligatoire en cloud

- KillerCoda ne possède pas les dossiers locaux.
- On supprime build: et on garde image:.
- Compose fait pull des images Docker Hub.
- Tester ensuite Eureka, Gateway et Keycloak via Traffic / Ports.

Conclusion

Docker transforme une architecture microservices en une stack reproductible : chaque service a son image, son container, son port, son réseau et ses dépendances.

- Dockerfile construit un microservice.
- docker build produit une image.
- docker run lance un container.
- Docker Hub partage l'image.
- Docker Compose orchestre toute la stack.